Unidad 1: Introducción al Proceso de Desarrollo de Software

1. Introducción General y Fundamentos del Proceso de Desarrollo

1.1 ¿Qué es el Desarrollo de Software?

El **Desarrollo de Software** (Software Development) es un proceso de ingeniería que aplica un enfoque sistemático, disciplinado y cuantificable para la **producción, operación y mantenimiento de software**. Más allá de la simple **codificación**, el desarrollo es un proceso intelectual y de gestión que busca transformar una **necesidad** o **problema** del mundo real en una solución tecnológica funcional, escalable y de calidad.

La Ingeniería de Software como Disciplina

La **Ingeniería de Software (IS)** es la disciplina que se encarga del Desarrollo de Software. Según el IEEE (**Institute of Electrical and Electronics Engineers - Instituto de Ingenieros Eléctricos y Electrónicos**), es la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software.

IEEE

El **IEEE (Institute of Electrical and Electronics Engineers)**, o Instituto de Ingenieros Eléctricos y Electrónicos, es la asociación profesional técnica más grande del mundo, dedicada al avance de la innovación tecnológica en diversas áreas, incluyendo la computación y la electrónica. En el campo de la Ingeniería de Software, el IEEE es una entidad clave porque **establece y publica estándares** reconocidos globalmente (como el estándar 1074 para el ciclo de vida del software o el 830 para la especificación de requisitos). Estos estándares buscan **formalizar, profesionalizar y asegurar la calidad** en las prácticas de la industria del software.

Para más información sobre sus actividades y publicaciones, se recomienda visitar el sitio oficial: IEEE Official Website.

Definiciones
Programación: Es la actividad de escribir el código fuente que permite implementar funcionalidades específicas dentro de un sistema. Representa una fase técnica dentro del proceso general de creación de software.

Desarrollo de Software: Es el conjunto completo de actividades que conforman el ciclo de vida del software (SDLC), desde la identificación de las necesidades del cliente y el análisis de requerimientos, hasta el diseño, la gestión del proyecto, la programación, las pruebas y el mantenimiento del sistema.

Diferencias
La programación se centra en la construcción técnica del software (escribir el código), mientras que el desarrollo de software abarca todo el proceso necesario para crear y mantener un

producto funcional y de calidad, incluyendo aspectos de análisis, diseño, gestión y aseguramiento de calidad.

Elementos Fundamentales de la Ingeniería de Software

La IS se enfoca en tres dimensiones interrelacionadas que definen el éxito de un proyecto:

1. **Producto (Product):** El software final entregable y los artefactos de trabajo que lo describen (documentación, diagramas, modelos). Su principal objetivo es cumplir con los requisitos y tener alta calidad técnica.
2. **Proceso (Process):** La serie de actividades (fases, tareas) y metodologías (Cascada, Ágil, Espiral) que se utilizan para construir el producto. El proceso define **cómo** se organiza el trabajo para crear el producto de manera eficiente.
3. **Personas (People):** El equipo de trabajo (roles, habilidades, comunicación) y los *stakeholders* (clientes, usuarios) que participan. La interacción humana y la gestión del talento son esenciales para el éxito.

Objetivos de Calidad del Software

Un software de calidad no es solo aquel que funciona, sino aquel que posee atributos medibles y deseables. Estos atributos son cruciales en la fase de Diseño y Pruebas:

- **Fiabilidad (Reliability):** La capacidad del software de operar sin fallos durante un período de tiempo específico bajo condiciones definidas.
- **Eficiencia (Efficiency) y Rendimiento:** La forma en que el sistema utiliza los recursos de hardware (tiempo de CPU, memoria, ancho de banda) para cumplir sus funciones rápidamente.
- **Mantenibilidad (Maintainability):** La facilidad con la que el software puede ser modificado (corregido, adaptado o mejorado). Este es el factor más importante ya que la fase de Mantenimiento consume la mayor parte del presupuesto.
- **Usabilidad (Usability):** La facilidad con la que los usuarios pueden aprender a operar el sistema y lo productivos que se vuelven al usarlo.

1.2 Fases Clásicas del Ciclo de Vida de Software (SDLC)

El **Ciclo de Vida de Desarrollo de Software (SDLC)** (Software Development Life Cycle) es un modelo conceptual utilizado en la gestión de proyectos que describe las etapas por las que pasa un proyecto de software. Este modelo se enfoca en la necesidad de una finalización rigurosa de cada fase antes de pasar a la siguiente.

El Estándar IEEE 1074: Proceso Formal

La estructura de las fases del SDLC a menudo se basa en el estándar **IEEE 1074**, el cual establece un conjunto de procesos, actividades y tareas para el desarrollo de software. Su objetivo es proporcionar una guía para la gestión, el control y la mejora del ciclo de vida, estandarizando los entregables esperados en cada etapa, lo que resulta fundamental para asegurar la **trazabilidad** y la **calidad** en proyectos grandes.

1. **Planificación (o Inicio):** Se establecen los fundamentos del proyecto.
    - **Actividades Clave:** Definición del alcance y las restricciones (Costo, Tiempo, Alcance/Calidad), y creación de un plan de proyecto de alto nivel.
    - **Estudio de Viabilidad (Feasibility Study):** Esta actividad determina si el proyecto es factible antes de invertir recursos significativos. Se evalúan cuatro dimensiones:
        - **Técnica:** ¿Tenemos (o podemos adquirir) la tecnología y las habilidades para construir el sistema?
        - **Económica:** ¿El costo de desarrollo es justificable por los beneficios que generará el sistema? (Cálculo del ROI - Retorno de la Inversión).
        - **Operacional:** ¿El sistema propuesto será compatible con los procesos de negocio y la cultura organizacional actual?
        - **Legal/Ética:** ¿El sistema cumple con las regulaciones, leyes de privacidad y normativas aplicables?
2. **Análisis de Requisitos:** El foco está en el *qué* debe hacer el sistema. Es la fase de descubrimiento, donde se extraen, se analizan y se documentan las necesidades del cliente.
    - **Documentación Formal (IEEE 830):** El resultado de esta fase es el **Documento de Especificación de Requisitos de Software (ERS)**. El estándar **IEEE 830** define la estructura y el contenido que debe tener este documento para que los requisitos sean presentados de forma coherente, completa y sin ambigüedades, sirviendo como contrato y guía para el desarrollo.
    - **Tipos de Requisitos:**
        - **Funcionales:** Describen el comportamiento o funciones que el sistema debe realizar (ej: "El sistema debe permitir a los usuarios iniciar sesión").
        - **No Funcionales:** Describen criterios de desempeño, calidad o restricciones (ej: seguridad, rendimiento, usabilidad, escalabilidad).

- **Características de un buen Requisito:** Para que un requisito sea útil, debe ser:
  - **No ambiguo:** Solo debe tener una interpretación.
  - **Consistente:** No debe contradecir a otro requisito.
  - **Completo:** Debe describir la funcionalidad o restricción por completo.
  - **Medible/Verificable:** Debe ser posible probar si el sistema lo cumple.

3. **Diseño:** El foco cambia al *cómo* se construirá el sistema para cumplir con los requisitos.
   - **Diseño de Alto Nivel (Arquitectura):** Se definen los componentes principales, sus interrelaciones, y las tecnologías a utilizar. Se toman decisiones fundamentales sobre la estructura general del sistema (ej: arquitectura de tres capas, microservicios, monolito).
   - **Diseño de Bajo Nivel (Detalle):** Se especifican las estructuras de datos, los algoritmos y la lógica interna a nivel de módulo, clase o función. Es el diseño que el programador traduce directamente a código.
4. **Implementación (Codificación):** Se traduce el diseño detallado en código fuente utilizando lenguajes de programación.
   - **Consideración Crítica:** Los modelos clásicos exigen que esta fase solo comience una vez que el diseño esté **completo y aprobado**. Se aplican estándares de codificación y se utiliza control de versiones (ej: Git) para gestionar los cambios.
5. **Pruebas (Testing/Verificación y Validación):** Se asegura la calidad del software.
   - **Verificación:** ¿Estamos construyendo el producto **correctamente**? (Se comprueba si el software cumple con el diseño y las especificaciones).
   - **Validación:** ¿Estamos construyendo el producto **correcto**? (Se comprueba si el software satisface las necesidades reales del cliente).
   - **Niveles de Prueba Comunes:**
     - **Pruebas Unitarias (Unit Testing):** Prueban la unidad más pequeña del código (función o clase) de forma aislada. Realizado generalmente por desarrolladores.
     - **Pruebas de Integración (Integration Testing):** Prueban que los módulos o componentes interactúen correctamente entre sí.

- **Pruebas de Sistema (System Testing):** Prueban el sistema completo para verificar el cumplimiento de todos los requisitos funcionales y no funcionales.
- **Pruebas de Aceptación (Acceptance Testing - UAT):** Pruebas finales realizadas por el cliente o usuario para confirmar que el sistema está listo para ser desplegado.

6. **Despliegue (Puesta en Producción):** El software se instala y se pone a disposición de los usuarios finales en su entorno operativo.
7. **Mantenimiento:** La fase más larga del ciclo de vida, comenzando tan pronto como el software es entregado.

Tipo de Mantenimiento	Objetivo Principal	Naturaleza del Cambio	Ejemplo de Tarea
Correctivo	Corregir fallos.	Reactivo. Arreglar errores y defectos que se manifiestan en producción.	Solucionar un *bug* que causa que un cálculo financiero sea incorrecto.
Adaptativo	Adaptar a cambios externos.	Reactivo/Preventivo. Ajustar el software a nuevos entornos técnicos o regulatorios.	Modificar la conexión a la base de datos tras una actualización del servidor o adaptar el sistema a una nueva ley de privacidad de datos.
Perfectivo	Mejorar funcionalidad y rendimiento.	Proactivo. Mejorar la usabilidad, la eficiencia y añadir nuevas características.	Optimizar consultas lentas para mejorar el tiempo de respuesta o añadir un nuevo informe solicitado por el usuario.
Preventivo	Aumentar la fiabilidad interna.	Proactivo. Mejorar la calidad del código interno para reducir la probabilidad de fallos futuros.	Refactorizar un módulo complejo para hacerlo más legible y fácil de mantener, o actualizar librerías internas para evitar obsolescencia.

1.4 Introducción al Concepto de Metodología

Una **Metodología de Desarrollo de Software** es un **conjunto estructurado de principios, prácticas y procesos** que proporcionan una guía para planificar, ejecutar y controlar un proyecto

de software. Su objetivo es organizar el trabajo de desarrollo de manera sistemática, predecible y eficiente, asegurando la calidad y el cumplimiento de los objetivos del proyecto.

Metodología vs. Marco de Trabajo (Framework)

Aunque a menudo se usan indistintamente, en el contexto de la Ingeniería de Software existe una sutil diferencia:

- **Metodología (Methodology):** Es un conjunto de **principios, valores y filosofías** que guían el trabajo. Responde al *por qué* y al *cómo* en un nivel alto (Ej: El Desarrollo Ágil es una filosofía metodológica).
- **Marco de Trabajo (Framework):** Es un **conjunto de herramientas, procesos y roles concretos y prescriptivos** que implementan una metodología. Es la aplicación práctica de esa filosofía (Ej: Scrum, Kanban son frameworks ágiles).

Propósito de una Metodología:

- **Reducir la complejidad:** Descompone el proyecto en tareas manejables y secuenciales.
- **Mejorar la comunicación:** Establece roles claros y artefactos estándar para el intercambio de información.
- **Estandarizar la calidad:** Asegura que se sigan pasos definidos y se produzcan entregables de calidad comprobable.
- **Aumentar la predictibilidad:** Permite la estimación de tiempos y costos con mayor precisión.

Componentes Clave: Toda metodología bien definida establece:

- **Fases y Tareas:** La secuencia lógica de actividades a realizar.
- **Roles y Responsabilidades:** La asignación de tareas dentro del equipo (ej: Analista de Sistemas, Diseñador de UX, Programador, Tester).
- **Artefactos y Entregables:** Los productos documentales o de código generados en cada fase (ej: Documento de Requisitos, Diagramas UML, Plan de Pruebas, Código Fuente).
- **Técnicas y Herramientas:** Los métodos específicos utilizados (ej: análisis estructurado, modelado orientado a objetos, herramientas de gestión de proyectos).

Tipos de Enfoques Metodológicos

Las metodologías se pueden clasificar en dos grandes grupos basados en su aproximación al cambio y a la planificación:

- **Enfoques Predictivos (Plan-Driven):**

- **Premisa:** Los requisitos se pueden definir completamente al inicio, y el plan no cambiará significativamente. El foco está en seguir el plan.
- **Características:** Secuenciales, énfasis en la documentación, bajo involucramiento del cliente durante la construcción.
- **Ejemplos:** Modelo en Cascada, V-Model.
- **Ideal para:** Proyectos con requisitos fijos, entornos estables o sistemas de alta criticidad (aeronáutica, medicina).

- **Enfoques Adaptativos (Change-Driven / Ágiles):**
  - **Premisa:** Los requisitos son volátiles y el cambio es bienvenido. El foco está en responder rápidamente a la evolución del mercado o del negocio.
  - **Características:** Iterativos e incrementales, énfasis en el software funcional, alta colaboración y *feedback* constante con el cliente.
  - **Ejemplos:** Scrum, Kanban, Extreme Programming (XP).
  - **Ideal para:** Proyectos donde los requisitos son complejos, inciertos o sujetos a cambios rápidos de mercado.

1.5 Grupos de Procesos del PMI (Gestión de Proyectos)

Es crucial diferenciar el Ciclo de Vida del Producto (SDLC) del Ciclo de Vida del Proyecto. Los modelos SDLC se centran en el **producto** (el software), mientras que el marco del **Project Management Institute (PMI)**, a través del PMBOK (Project Management Body of Knowledge), se centra en la **gestión** del proyecto (el esfuerzo organizacional).

Cada uno de estos Grupos de Procesos del PMI, a su vez, está compuesto por una serie de **Áreas de Conocimiento** (como Gestión del Alcance, Gestión del Tiempo, Gestión de Riesgos, etc.) y múltiples **subprocesos** que detallan las entradas, herramientas, técnicas y salidas necesarias para gestionar el proyecto de forma completa.

Los Grupos de Procesos del PMI se aplican en cada fase del SDLC para asegurar que el proyecto se mantenga dentro del alcance, tiempo y presupuesto.

Para más información, puede consultar la documentación oficial del PMI: Project Management Institute.

Grupo de Proceso PMI	Propósito	Relación con el SDLC
Inicio (Initiating)	Definir un nuevo proyecto o una nueva fase, obteniendo la autorización necesaria.	Se alinea con la fase de **Planificación (o Inicio)** del SDLC. Determina la viabilidad y el alcance de alto nivel.

Planificación (Planning)	Establecer el alcance, refinar los objetivos y definir el curso de acción necesario para alcanzar los objetivos.	Se ejecuta durante el **Análisis de Requisitos** y el **Diseño** del SDLC, creando planes detallados de trabajo, tiempo y recursos.
Ejecución (Executing)	Realizar el trabajo definido en el plan de gestión del proyecto para satisfacer los requisitos.	Corresponde principalmente a las fases de **Diseño, Implementación y Pruebas** técnicas del SDLC.
Seguimiento y Control (Monitoring & Controlling)	Rastrear, revisar y regular el progreso y el desempeño del proyecto; identificar áreas donde el plan requiere cambios; e iniciar dichos cambios.	Se aplica de forma **continua** a lo largo de todas las fases del SDLC, asegurando que el diseño y la codificación sigan el plan.
Cierre (Closing)	Finalizar todas las actividades para cerrar formalmente el proyecto o una fase del proyecto.	Coincide con el **Despliegue** y la transferencia a **Mantenimiento**, incluyendo la documentación de lecciones aprendidas.